HTTP-Server例程

HTTP-Server例程
- 例程讲解
  - 简介
  - 关键部分代码
  - 流程图
- HTTP 服务端
  - HTTP 请求
  - HTML 页面
  - HTTP 状态码

例程讲解

简介

本节我们使用K230作为HTTP服务器使用。

我们打开例程代码，修改main函数中的Connect_WIFI函数的参数为SSID（WIFI的名称）和KEY（WIFI的密码）

```
ip = Connect_WIFI("[SSID]", "[KEY]")
```

修改好之后我们点击运行按钮，可以看到串行中断输出如下内容

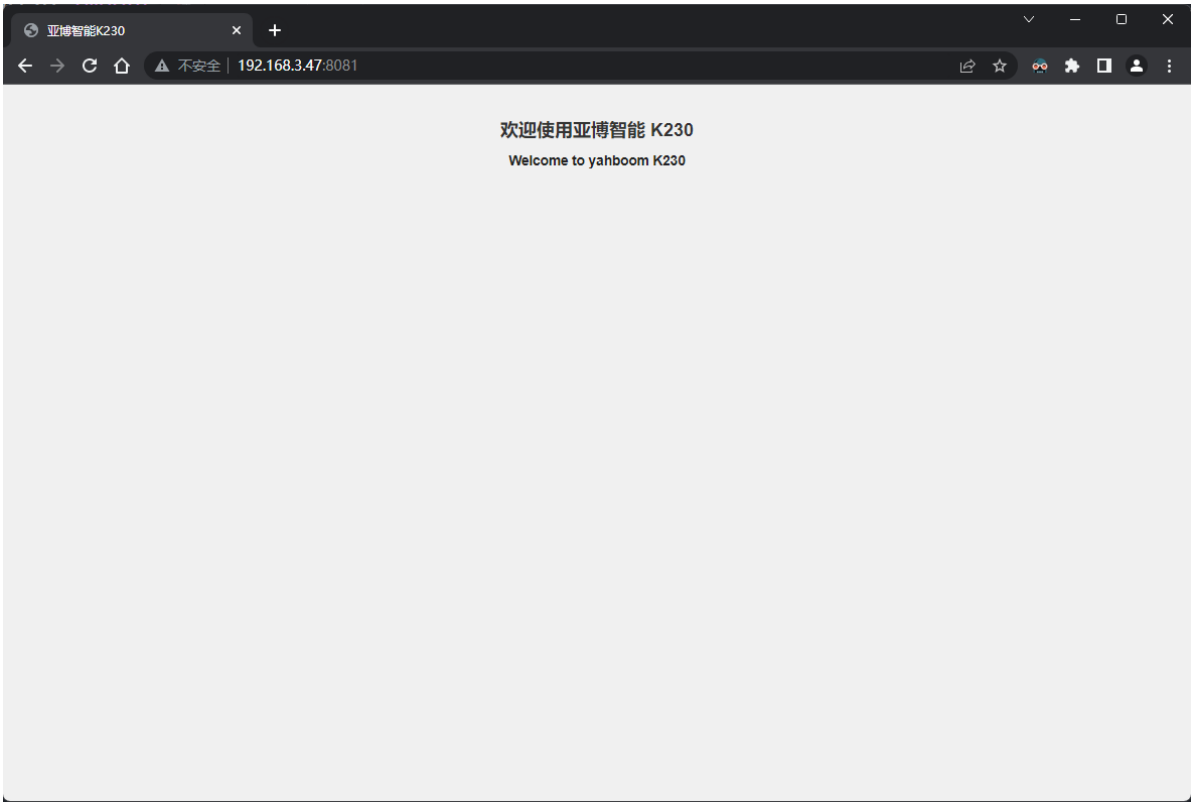
```
True
('192.168.3.47', '255.255.255.0', '192.168.3.1', '192.168.3.1')
Listening, connect your browser to http://192.168.3.47:8081/
```

第一行的True表示WIFI连接成功

第二行输出的是连接的WIFI的信息

第三行表示HTTP服务器启动成功了，我们可以复制这个地址，粘贴到浏览器的地址栏，按下回车键

可以看到浏览器中显示这个页面，则说明启动成功了



关键部分代码

完整代码请参考 【源码汇总 / 11.Network / 05.http / http_server.py】

```
def main(micropython_optimize=True):
    # 连接WiFi并获取IP Connect to WiFi and get IP
    ip = Connect_WIFI("Yahboom", "yahboom890729")

    # 创建socket对象 Create socket object
    s = socket.socket()
    # 获取地址信息 Get address information
    ai = socket.getaddrinfo("0.0.0.0", 8081)
    addr = ai[0][-1]
    # 设置socket选项 Set socket options
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    # 绑定地址 Bind address
    s.bind(addr)
    # 开始监听 Start listening
    s.listen(5)
    print("Listening, connect your browser to http://%s:8081/" % (ip))

    counter = 0
    # 主循环处理客户端连接 Main loop to handle client connections
    while True:
        # 接受客户端连接 Accept client connection
        res = s.accept()
        client_sock = res[0]
        client_addr = res[1]
        print("Client address:", client_addr)
        # 设置非阻塞模式 Set non-blocking mode
        client_sock.setblocking(False)
        # 获取客户端流 Get client stream
        client_stream = client_sock if micropython_optimize else
client_sock.makefile("rwb")
```

```
        # 读取HTTP请求头 Read HTTP request headers
        while True:
            h = client_stream.read()
            if h is None or h==b'':
                continue
            print(h)
            if h.endswith(b'\r\n\r\n'):
                break
            os.exitpoint()

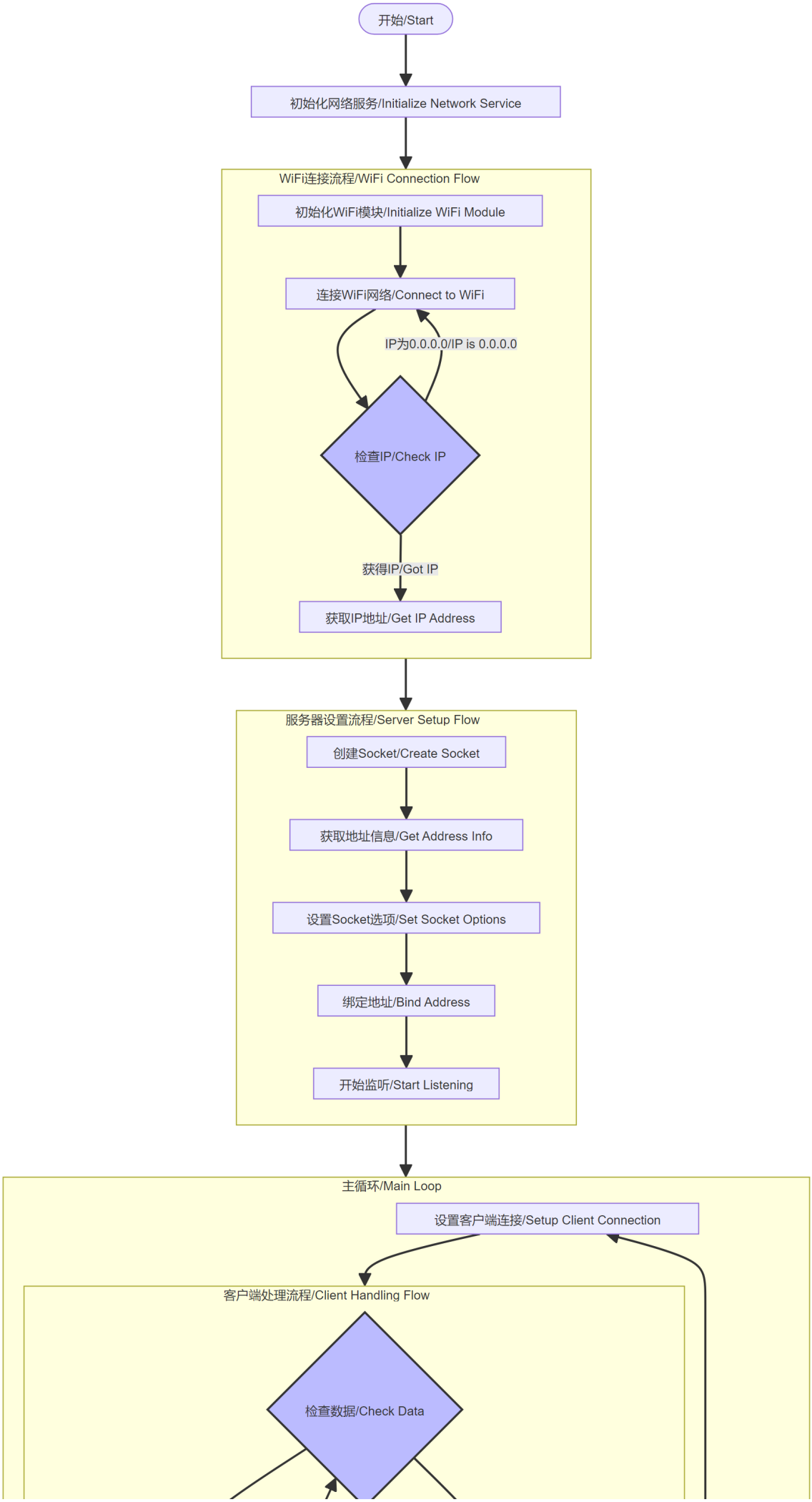
        # 发送响应内容 Send response content
        client_stream.write(CONTENT)
        # 关闭客户端连接 Close client connection
        client_stream.close()

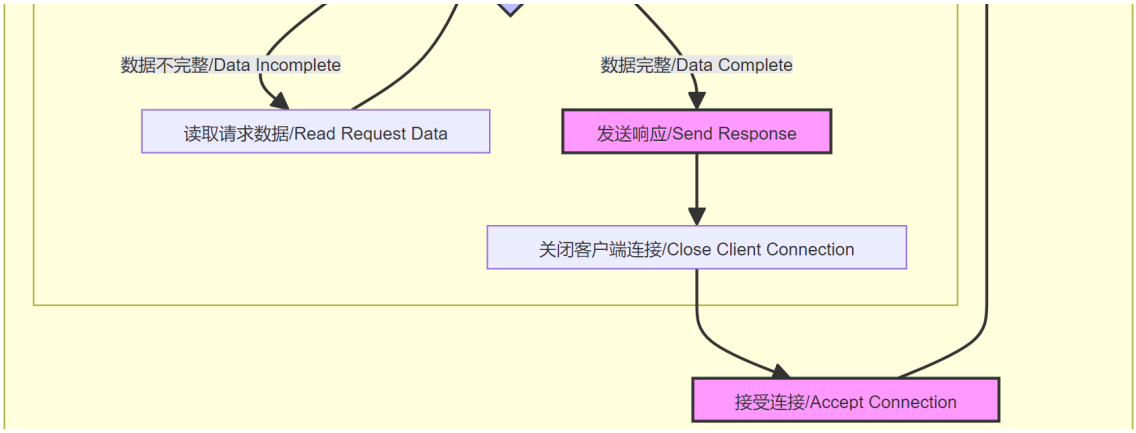
# 运行主函数 Run main function
main()
```

其中CONTENT部分是我们浏览器中显示的页面代码，使用的是HTML语言

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>亚博智能K230</title>
        <style>
            body {
                font-family: Arial, sans-serif;
                margin: 0;
                padding: 20px;
                background-color: #f0f0f0;
            }
            h1 {
                color: #333;
                text-align: center;
            }
            h2 {
                color: #222;
                text-align: center;
            }
        </style>
    </head>
    <body>
        <h1>欢迎使用亚博智能 K230</h1>
        <h2>Welcome to yahboom K230</h2>
    </body>
</html>
```

流程图


开始/Start
初始化网络服务/Initialize Network Service
WiFi连接流程/WiFi Connection Flow
初始化WiFi模块/Initialize WiFi Module
连接WiFi网络/Connect to WiFi
IP为0.0.0.0/IP is 0.0.0.0
检查IP/Check IP
获得IP/Got IP
获取IP地址/Get IP Address
服务器设置流程/Server Setup Flow
创建Socket/Create Socket
获取地址信息/Get Address Info
设置Socket选项/Set Socket Options
绑定地址/Bind Address
开始监听/Start Listening
主循环/Main Loop
设置客户端连接/Setup Client Connection
客户端处理流程/Client Handling Flow
检查数据/Check Data



HTTP 服务端

这里我们来简单描述一下HTTP服务端

HTTP 请求

> HTTP(超文本传输协议)是一个用于在网络上传输数据的应用层协议。

我们平时使用浏览器访问网页，我们在网页上进行的点击或者任何操作，实际上就是通过浏览器向HTTP服务器发送请求，然后HTTP服务器返回数据（数据可以是图片、视频、或者HTML页面等任何类型的数据，具体返回什么是根据你请求的操作决定的）。我们本节教程实现的 HTTP服务器做的事情就是 接收http请求，返回一个HTML页面数据。

HTML 页面

HTML页面是一种可以被浏览器识别并解析的描述文件。可以简单理解为我们平时看到的网页就是由HTML语言编写的。本节教程中我们的CONTENT变量中的内容就是一个简单的HTML代码。当浏览器接收到HTTP服务器发送的HTML文件后，就会把HTML文件的内容解析成我们看到的好看的网页。

HTTP 状态码

HTTP状态码是服务器对客户端请求的处理结果的标识，分为5类：

1xx (信息性状态码)

- 100 Continue: 继续发送请求
- 101 Switching Protocols: 切换协议

2xx (成功状态码)

- 200 OK: 请求成功
- 201 Created: 创建成功
- 204 No Content: 成功但无返回内容
- 206 Partial Content: 部分内容请求成功

3xx (重定向状态码)

- 301 Moved Permanently: 永久重定向
- 302 Found: 临时重定向
- 304 Not Modified: 资源未修改
- 307 Temporary Redirect: 临时重定向

4xx (客户端错误)

- 400 Bad Request: 请求语法错误
- 401 Unauthorized: 未授权
- 403 Forbidden: 禁止访问
- 404 Not Found: 资源不存在
- 405 Method Not Allowed: 方法不允许
- 429 Too Many Requests: 请求过多

5xx (服务器错误)

- 500 Internal Server Error: 服务器内部错误
- 502 Bad Gateway: 网关错误
- 503 Service Unavailable: 服务不可用
- 504 Gateway Timeout: 网关超时

> 本节教程中的CONTENT的开头
>
> b"""\
> HTTP/1.0 200 OK\r\n\
> Content-Type: text/html; charset=utf-8\r\n\
> \r\n\
>
> 这一段就是HTTP响应报文的头部，里面的200就代表着【200 OK: 请求成功】